

Informe de Arquitectura de Software

Plataforma Web de Cursos Masivos Abiertos en Línea (MOOC)

Entrega 1 — Backend y Workers

Monolito Modular en Go, Workers Asíncronos, Persistencia Relacional y Observabilidad

Proyecto:	Plataforma MOOC (Massive Open Online Courses)
Programa / Asignatura:	ISIS4426 — Arquitectura Cloud / Soluciones Software
Organización Operadora:	Institución Operadora de Plataforma MOOC
Alcance de este informe:	Entrega 1 — Backend y Workers (Issues #1 a #30)
Versión de este documento:	1.1.0
Fecha de emisión:	Septiembre de 2026
Estado del documento:	Vigente — Entrega 1 completada
Repositorio de referencia:	github.com/ISIS4426-2026/Plataforma-MOOC (rama main, commit f8450cc)
Equipo responsable:	Equipo de Arquitectura e Ingeniería MOOC

Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
Confidencial — Uso Académico e Institucional

Control de Versiones del Documento

Versión	Fecha	Autor(es)	Revisor(es)	Descripción
1.0.0	14/09/2026	Equipo de Arquitectura e Ingeniería MOOC	Pendiente de asignar	Emisión de la línea base de arquitectura para la Entrega 1 (Issues #1 a #30).
1.1.0	15/09/2026	Equipo de Arquitectura e Ingeniería MOOC	Pendiente de asignar	Incorpora diagramas de arquitectura (C4, modelo de datos, secuencia) y el plan de entregas siguientes.

Este informe documenta la arquitectura de software de la Plataforma MOOC tal como fue implementada y verificada en la Entrega 1 del proyecto (Issues #1 a #30, milestone "Entrega 1 - Backend y Workers"), y presenta el plan de evolución previsto para las siguientes entregas del curso.

Tabla de Contenido

1. Resumen Ejecutivo

2. Contexto de Negocio, Objetivos y Alcance

- 2.1 Propósito y Visión del Sistema
- 2.2 Roles Globales y Modelo de Actores
- 2.3 Jerarquía Académica y Tipos de Recursos
- 2.4 Guardrails y Principios Inquebrantables de Arquitectura
- 2.5 Alcance de la Entrega 1

3. Atributos de Calidad y Requerimientos No Funcionales (ASRs)

- 3.1 Escalabilidad y Concurrencia
- 3.2 Disponibilidad y Resiliencia
- 3.3 Seguridad e Integridad Transaccional
- 3.4 Rendimiento y Latencia
- 3.5 Observabilidad y Auditabilidad
- 3.6 Mantenibilidad y Desacoplamiento

4. Registro de Decisiones de Arquitectura (ADRs)

5. Vistas de Arquitectura (Modelo C4 / ISO/IEC 42010)

- 5.1 Vista de Contexto del Sistema
- 5.2 Vista de Contenedores y Topología
- 5.3 Vista de Componentes y Modularidad del Backend
- 5.4 Vista de Información y Datos
- 5.5 Vista de Despliegue e Infraestructura
- 5.6 Vista Dinámica: Flujos Críticos de Ejecución

6. Estrategia de Seguridad, Gobierno y Cumplimiento

7. Estrategia de Resiliencia, Concurrencia y Recuperación ante Desastres (DRP)

8. Estrategia de Observabilidad, Telemetría y Monitoreo

9. Estado de Implementación — Entrega 1

10. Plan de Evolución Arquitectónica y Roadmap Técnico

11. Glosario de Términos

12. Referencias

1. Resumen Ejecutivo

Este informe documenta la arquitectura de software de la Plataforma MOOC, un sistema de cursos masivos abiertos en línea operado por una única entidad, en el estado correspondiente a la Entrega 1 del proyecto: backend y capa de workers funcionales, sin frontend, verificados mediante pruebas automáticas contra el sistema desplegado en Docker Compose.

La solución adopta un Monolito Modular en Go con Arquitectura Hexagonal, procesamiento asíncrono desacoplado mediante Redis y Asynq, y persistencia transaccional en PostgreSQL. La Entrega 1 completó los 30 issues planificados para este hito (milestone "Entrega 1 - Backend y Workers"), organizados en cinco frentes:

- **Identidad y control de acceso** — registro público con verificación por correo (Mailpit), sesiones opacas revocables, recuperación de contraseña y límites de tasa por endpoint.
- **Administración y auditoría** — gestión de roles, suspensión/reactivación, protección del último administrador activo y auditoría inmutable a nivel de base de datos.
- **Autoría académica y publicación** — jerarquía de cuatro niveles, reordenamiento atómico libre de colisiones, validación estructural acumulativa y publicación de versiones inmutables con identificadores estables.
- **Procesamiento asíncrono** — cola distribuida Redis + Asynq, idempotencia ante doble entrega, reintentos con backoff exponencial y Dead-Letter Queue.
- **Observabilidad transversal** — OpenTelemetry, métricas Prometheus y logs JSON correlacionados por request_id y trace_id.

Cada uno de estos frentes cuenta con evidencia de verificación automática: 216 aserciones sobre tres colecciones Postman/Newman (Identidad, Administración y Autoría), ejecutadas contra el sistema real en contenedores (cero mocks), junto con 37 archivos de prueba unitaria e integración y la verificación reversible de las cinco migraciones del esquema relacional.

Los módulos de multimedia, evaluación de quizzes, progreso e insignias, y el frontend de consumo, están fuera del alcance de esta entrega por diseño — se documentan en el Plan de Evolución (Sección 10) junto con la distribución prevista para las próximas entregas del curso.

2. Contexto de Negocio, Objetivos y Alcance

2.1 Propósito y Visión del Sistema

La plataforma resuelve la necesidad de ofrecer educación en línea masiva, asíncrona y autodirigida sin depender de plataformas SaaS de terceros, garantizando privacidad de datos, control de versiones de contenido académico, evaluación automatizada y emisión verificable de insignias de aprobación.

- Capacidad nominal objetivo: hasta 50.000 usuarios registrados activos en catálogo.
- Concurrencia pico objetivo: hasta 2.000 usuarios concurrentes sin degradación de SLA.
- Modelo operacional: contenedores Docker / Docker Compose, con preparación nativa para orquestación en la nube (Kubernetes / ECS) en una etapa posterior.

2.2 Roles Globales y Modelo de Actores

El sistema implementa control de acceso basado en roles (RBAC) con tres roles no intercambiables, verificado en `internal/http/middleware/authorize.go` y cubierto por pruebas automáticas en `internal/http/middleware/authorize_test.go`:

- **Administrador** — control administrativo de cuentas, roles, suspensión/reactivación y sesiones; consulta de auditoría inmutable; único rol facultado para crear y promover cuentas de profesores; protección intransigente contra la eliminación o suspensión del último administrador activo.
- **Profesor** — creación, edición, estructuración y previsualización de cursos; reordenamiento de módulos, unidades y recursos; solicitud de publicación de versiones inmutables y despublicación temporal.
- **Estudiante** — autorregistro sujeto a verificación por correo; exploración de catálogo público. El consumo de contenido (inscripción, reproducción, quizzes, progreso) se incorpora en próximas entregas, ver Sección 10.

2.3 Jerarquía Académica y Tipos de Recursos

La estructura de contenidos sigue una jerarquía formal de cuatro niveles, materializada en el esquema relacional (`migrations/000001_init_schema.up.sql`) y en el modelo de dominio (`internal/domain/course.go`):

Curso → Módulo → Unidad → Recurso

- **Curso**: agregado raíz de autoría. Metadatos, autor responsable, versión numérica y estado de ciclo de vida (`draft`, `published`, `unpublished`).
- **Módulo**: agrupador temático ordenado dentro de la versión del curso (`position`).
- **Unidad**: subdivisión didáctica ordenada dentro de un módulo (`position`).
- **Recurso**: unidad atómica de aprendizaje. Define visibilidad, obligatoriedad para aprobación y descargabilidad (`allow_download`).

Se soportan 10 tipos de recurso: text, image, video, audio, pdf, presentation, downloadable, iframe, external_link y quiz. El contenido textual se persiste como Markdown extendido canónico; los binarios se conservan exclusivamente en almacenamiento de objetos (ver ADR-03, Sección 4).

2.4 Guardrails y Principios Inquebrantables de Arquitectura

Los siguientes cinco principios son invariantes no negociables del sistema. Se listan aquí como referencia rápida; su fundamentación y consecuencias técnicas completas se documentan como Decisiones de Arquitectura en la Sección 4 (ADR-03, ADR-04, ADR-06, ADR-07 y ADR-08, respectivamente).

- **Quiz Key Secrecy:** la clave de respuestas correctas nunca viaja al cliente; la calificación ocurre exclusivamente en el servidor.
- **Progreso verificado en servidor:** se rechazan y auditan los porcentajes de avance inyectados por el cliente.
- **Inmutabilidad de publicación:** una versión published no admite modificaciones directas (HTTP 409); para editar se exige despublicación temporal.
- **Identificadores estables (stable_id):** persisten entre versiones para preservar el progreso histórico del estudiante.
- **Cero binarios en base relacional:** todo binario reside en S3/MinIO y se accede vía URLs prefirmadas.

2.5 Alcance de la Entrega 1

El milestone "Entrega 1 - Backend y Workers" define el alcance de esta entrega como backend y capa de workers funcionales, sin frontend, verificados mediante Postman. Los 30 issues planificados para este hito están completos:

Dominio Funcional	Estado en la Entrega 1
Identidad y Acceso	Implementado y verificado E2E (registro, verificación por correo, login, sesiones revocables, recuperación de contraseña, rate limiting).
Administración	Implementado y verificado E2E (roles, suspensión, protección del último administrador, auditoría inmutable).
Autoría de Cursos y Publicación	Implementado y verificado E2E (jerarquía de 4 niveles, validación acumulativa, versiones inmutables, stable_id).
Workers Asíncronos	Implementado (idempotencia, reintentos con backoff, Dead-Letter Queue) para tareas genéricas; verificado mediante script de demostración dirigido.
Observabilidad	Implementado (OpenTelemetry, métricas Prometheus, logs JSON estructurados).

Dominio Funcional	Estado en la Entrega 1
Calidad y QA	Plan de pruebas, datos sintéticos, 3 colecciones Postman (216 aserciones, 100% éxito) y 37 pruebas unitarias/integración.

Multimedia, evaluación de quizzes, progreso, insignias, catálogo con inscripción y el frontend de consumo se planifican para las Entregas 2 y 3, conforme al Plan de Evolución (Sección 10).

3. Atributos de Calidad y Requerimientos No Funcionales (ASRs)

Los Requerimientos de Atributos de Calidad (ASRs) guían las decisiones estructurales del proyecto y se verifican contra el código y las pruebas del repositorio.

3.1 Escalabilidad y Concurrencia

- **Statelessness estricto:** la API y los Workers no conservan estado en memoria de proceso ni en disco local; cualquier petición puede ser atendida por cualquier réplica.
- **Sesiones distribuidas:** se validan con tokens opacos cuyo hash se consulta en PostgreSQL/Redis, sin afinidad de sesión.
- **Particionamiento de tareas:** las operaciones de alta latencia se desacoplan mediante la cola Asynq/Redis.
- **Pool de conexiones a PostgreSQL:** 25 conexiones abiertas, 25 ociosas, 5 minutos de vida máxima.

3.2 Disponibilidad y Resiliencia

- RPO (Recovery Point Objective) $\leq$ 15 minutos, mediante replicación continua de base de datos y respaldos de estado.
- RTO (Recovery Time Objective) $\leq$ 4 horas, mediante reconstrucción completa por contenedores Docker y migraciones reversibles.
- Tolerancia a fallos en workers: reintentos con backoff exponencial (hasta 3 intentos) y derivación a Dead-Letter Queue con alerta DLQ_JOB_FAILED — implementado y verificado en `internal/worker/dlq_test.go` y mediante el script `make demo-segment4` (ver diagrama de secuencia, Sección 5.6).

3.3 Seguridad e Integridad Transaccional

- **Hashing de contraseñas:** bcrypt con costo adaptativo (`internal/auth/password.go`).
- **Rate limiting:** distribuido en Redis por IP y tipo de endpoint (login: 10/min, registro: 50/hora).
- **Sanitización de texto libre:** contra inyección de HTML/JavaScript (XSS).
- **Inmutabilidad forense:** trigger PL/pgSQL que rechaza UPDATE/DELETE sobre `audit_logs` — verificado en `internal/postgres/audit_repository_test.go`.

3.3.1 Transporte de sesión y CSRF

El sistema devuelve el token de sesión en el cuerpo de la respuesta de login (esquema Bearer), no en una cookie. La verificación de encabezados Origin/Referer en `internal/http/middleware/csrf.go` se mantiene como capa adicional de defensa en profundidad para integraciones que sí usen cookies en entregas futuras.

3.4 Rendimiento y Latencia

- ETag y validación condicional de caché: SHA-256(id + updated_at) en entidades maestras; respuestas 304 ante coincidencia con If-None-Match — verificado en `internal/postgres/etag_test.go`.
- Paginación por cursores, evitando OFFSET en colecciones grandes.

3.5 Observabilidad y Auditabilidad

- Trazabilidad de extremo a extremo mediante X-Request-ID y trace_id de OpenTelemetry, inyectados en cada línea de log JSON estructurado.
- Métricas Prometheus en `/api/v1/metrics` (API) y `:9090/metrics` (Worker) — verificado en `internal/observability/observability_test.go`.

3.6 Mantenibilidad y Desacoplamiento

- Arquitectura Hexagonal: `internal/domain` no depende de `net/http`, `gin/fiber` ni de drivers de nube o base de datos.
- Contrato formal en OpenAPI 3.1 (`api/openapi.yaml`), verificado en CI con Spectral.

4. Registro de Decisiones de Arquitectura (ADRs)

Esta sección documenta las nueve decisiones de arquitectura que rigen la Entrega 1, cada una verificada contra el código y las pruebas del repositorio.

ADR-01: Monolito Modular en Go con Arquitectura Hexagonal

Contexto. El sistema requiere desarrollo rápido para el horizonte del MVP (13 semanas, 4 desarrolladores) sin la sobrecarga operativa de microservicios, garantizando desacoplamiento para escalar a futuro.

Decisión. Adoptar un Monolito Modular en Go bajo Arquitectura Hexagonal (Puertos y Adaptadores). `internal/domain` encapsula las reglas puras sin dependencias de infraestructura.

Consecuencias. Alta velocidad de compilación, despliegue unificado, pruebas unitarias aisladas mediante fakes de puertos.

Verificación. La estructura de paquetes del repositorio (`internal/domain`, `internal/http`, `internal/postgres`, `internal/cache`) corresponde exactamente a lo decidido (ver Sección 5.3).

ADR-02: Procesamiento Asíncrono con Workers Independientes (Asynq/Redis)

Contexto. Tareas como transcodificación, procesamiento de PDF y notificaciones por correo tienen duración no predecible y bloquearían los hilos de la API HTTP.

Decisión. Separar la ejecución en dos binarios independientes: `cmd/api` y `cmd/worker`, comunicados por Redis mediante asynq.

Consecuencias. La API permanece rápida; los workers escalan de forma independiente según la carga de procesamiento.

Verificación. `cmd/api/main.go` y `cmd/worker/main.go` existen como binarios separados; `internal/worker` cuenta con pruebas de backoff, DLQ, idempotencia y métricas.

ADR-03: Segregación de Binarios en Object Storage S3/MinIO

Contexto. Almacenar archivos multimedia en la base relacional degrada índices, infla respaldos y satura memoria.

Decisión. Regla estricta de cero binarios en PostgreSQL. Los archivos residirán exclusivamente en S3/MinIO, con transferencia directa mediante URLs prefirmadas (TTL 24h).

Consecuencias. Rendimiento óptimo en PostgreSQL, descarga de tráfico en la API, compatibilidad con CDN.

Verificación. MinIO está provisionado en `docker-compose.yml` y el esquema no contiene columnas BLOB/BYTEA (verificado por `migrations/migrations_test.go`). El flujo de carga se implementa en la Entrega 2 (ver Sección 10).

ADR-04: Identificadores Estables (stable_id) para Continuidad de Progreso

Contexto. Cuando un profesor publica una actualización de un curso existente, los estudiantes inscritos no deben perder su historial de avance.

Decisión. Cada entidad académica posee id (específico de versión) y stable_id (permanente entre versiones).

Consecuencias. Los profesores pueden reestructurar cursos sin corromper el avance académico de los alumnos.

Verificación. Verificado E2E — el caso de prueba "Verificación de identificador estable preservado" (colección de Autoría) confirma el comportamiento tras reordenamiento y eliminación de recursos.

ADR-05: Reordenamiento Atómico con Restricciones Diferidas

Contexto. La jerarquía exige posiciones contiguas sin huecos ni duplicados entre hermanos; un reordenamiento secuencial tradicional causaría colisiones transitorias de claves únicas.

Decisión. Algoritmo domain.Reposition que recalcula todas las posiciones en una sola pasada densa (0..n-1), con restricciones UNIQUE(...) DEFERRABLE INITIALLY DEFERRED en PostgreSQL.

Consecuencias. Se eliminan las condiciones de carrera y fallas por colisión de índices únicos a mitad de transacción.

Verificación. internal/domain/ordering.go y ordering_test.go; migración 000003_academic_ordering.up.sql aplica las restricciones diferidas.

ADR-06: Idempotencia en Capa HTTP y Colas de Tareas

Contexto. Clientes inestables pueden reenviar peticiones duplicadas; las colas garantizan entrega al menos una vez, pudiendo duplicar la entrega al worker.

Decisión. En la API: cabecera Idempotency-Key almacenada en Redis. En el Worker: deduplicación en Redis.

Consecuencias. Consistencia total frente a fallas de red y reintentos automáticos.

Verificación. Casos "Registro con Idempotency-Key", "Mismo POST, misma clave" y "Misma clave con cuerpo diferente" en la colección de Identidad (ver diagrama de secuencia, Sección 5.6).

ADR-07: Validación Estructural Acumulativa e Inmutabilidad de Publicación

Contexto. Validar un curso campo por campo obliga a ciclos frustrantes de fallo-corrección; modificar un curso en vivo corrompe la experiencia de estudiantes activos.

Decisión. El validador de publicación recorre toda la jerarquía y acumula los incumplimientos en un arreglo ValidationErrors, retornado en un único HTTP 422. Un curso publicado pasa a estado inmutable; cualquier cambio exige despublicación temporal.

Consecuencias. Mejor experiencia de autoría y protección contra inconsistencias curriculares.

Verificación. `internal/course/publish_test.go` y los casos "Intento de publicación con errores", "Publicación exitosa" y "Rechazo de edición sobre versión publicada (409)" (ver diagrama de secuencia, Sección 5.6).

ADR-08: Inmutabilidad de Auditoría Garantizada a Nivel de Base de Datos

Contexto. Los registros de auditoría deben ser legalmente vinculantes y a prueba de manipulaciones, incluso si la aplicación es comprometida.

Decisión. Trigger PL/pgSQL (migración `000004_audit_immutability`) que rechaza incondicionalmente cualquier UPDATE o DELETE sobre `audit_logs`.

Consecuencias. La pista de auditoría es estrictamente append-only.

Verificación. `internal/postgres/audit_repository_test.go` ejercita el rechazo del trigger.

ADR-09: Observabilidad Nativa Correlacionada

Contexto. En un sistema con workers asíncronos, diagnosticar incidentes requiere correlacionar el evento de la API con el trabajo procesado por el worker.

Decisión. Adoptar OpenTelemetry para trazas y métricas, con logs JSON estructurados (log/slog). El middleware inyecta X-Request-ID y `trace_id`.

Consecuencias. Capacidad de rastrear transacciones completas filtrando por `request_id` o `trace_id`.

Verificación. `internal/observability/observability_test.go`.

5. Vistas de Arquitectura (Modelo C4 / ISO/IEC 42010)

Esta sección documenta las vistas C4 del sistema tal como fue implementado en la Entrega 1: contexto, contenedores, componentes, información y despliegue, junto con los flujos dinámicos críticos.

5.1 Vista de Contexto del Sistema (C4 — Nivel 1)

Tres tipos de usuario (Estudiante, Profesor, Administrador) interactúan con la Plataforma MOOC vía HTTPS/REST API. En esta entrega, el sistema se integra activamente con un servidor SMTP (Mailpit) para correos transaccionales; el almacenamiento de objetos (MinIO) está aprovisionado en la infraestructura pero su flujo de carga se activa en la Entrega 2.

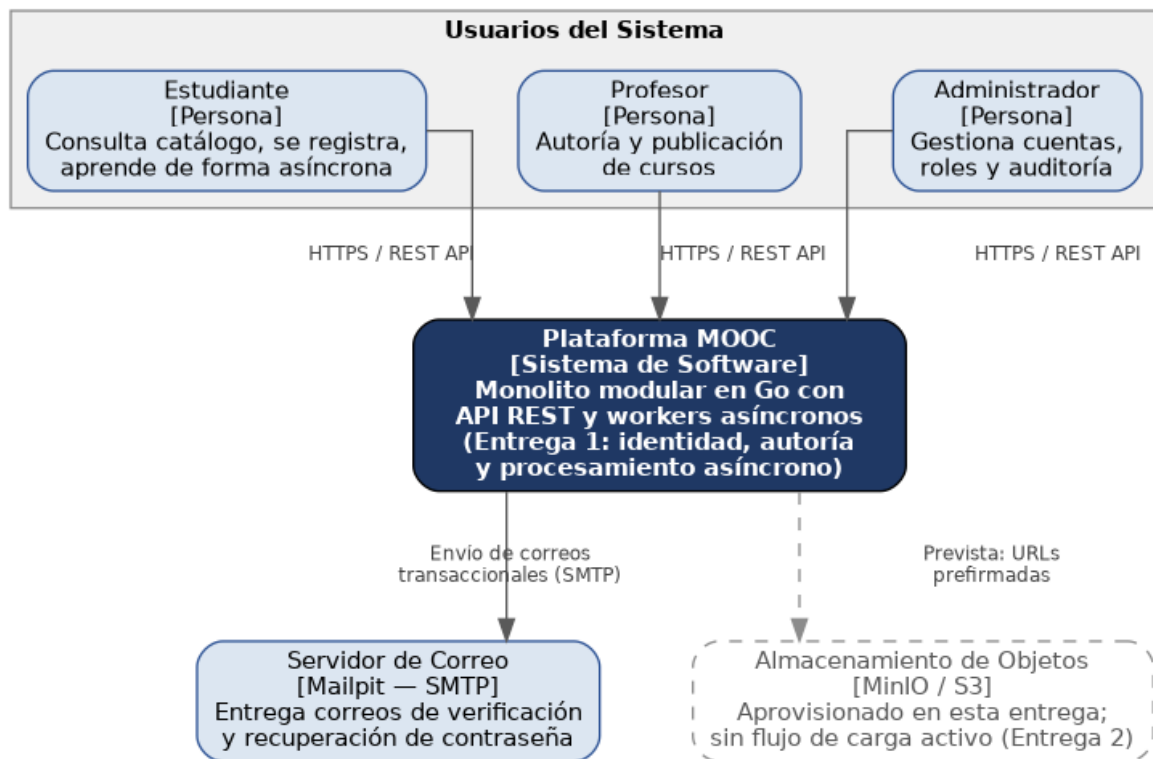


Figura 1 — Vista de Contexto del Sistema (C4 Nivel 1).

5.2 Vista de Contenedores y Topología (C4 — Nivel 2)

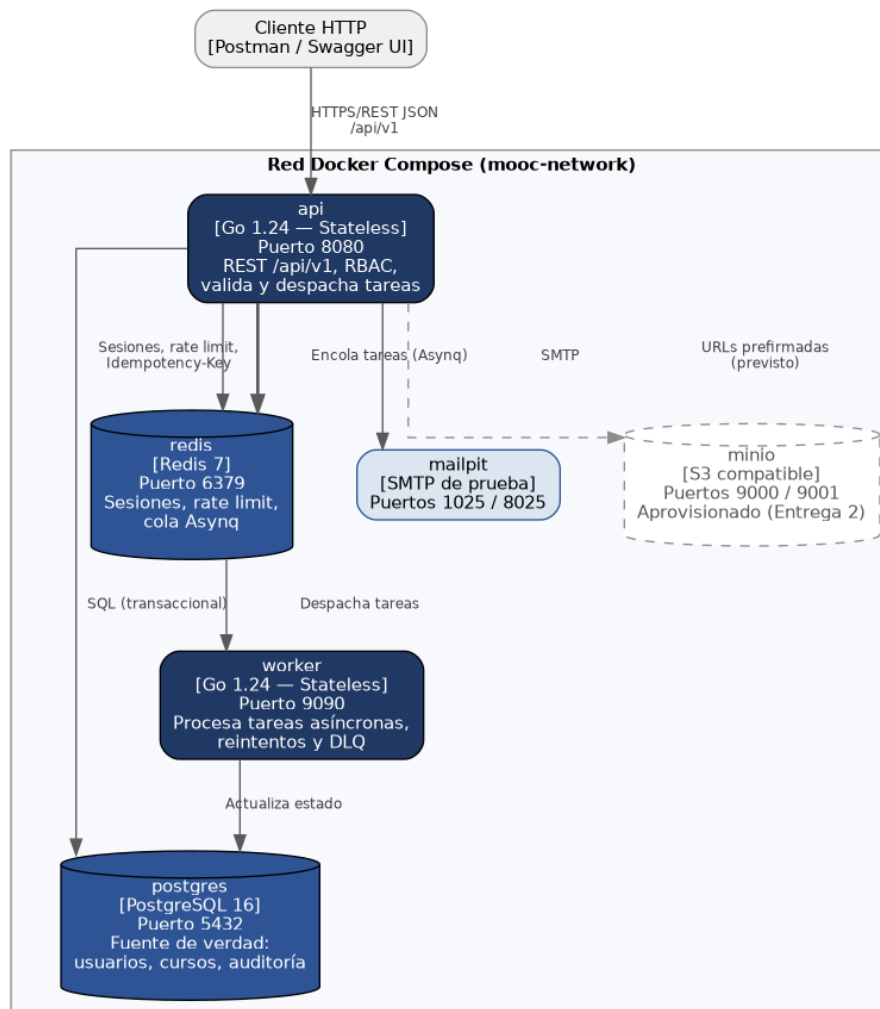


Figura 2 — Vista de Contenedores y Topología de Red en Docker Compose (C4 Nivel 2).

Contenedor	Imagen base	Puerto	Función
api	golang:1.24 → alpine (multi-stage)	8080	REST JSON /api/v1, Swagger UI, RBAC, métricas.
worker	golang:1.24 → alpine (multi-stage)	9090	Procesador Asynq, reintentos, DLQ, métricas.
postgres	postgres:16-alpine	5432	Persistencia relacional, triggers de inmutabilidad.
redis	redis:7-alpine	6379	Sesiones, rate limiting, cola Asynq.
mailpit	axllent/mailpit:latest	1025 / 8025	SMTP de prueba, verificación y recuperación.
minio	minio/minio:latest	9000 / 9001	Almacenamiento S3-compatible, aprovisionado para la Entrega 2.

5.3 Vista de Componentes y Modularidad del Backend (C4 — Nivel 3)

El backend implementa Arquitectura Hexagonal: el dominio no conoce PostgreSQL, Redis, HTTP ni S3. Estructura de paquetes verificada contra el árbol real del repositorio:

- cmd/ — puntos de entrada ejecutables (cmd/api/main.go, cmd/worker/main.go).
- internal/domain/ — entidades puras (admin, audit, badge, course, idempotency, ordering, ports, progress, quiz, ratelimit, session, token, user, validation) e interfaces de puertos.
- internal/auth/, internal/admin/, internal/course/, internal/structure/ — casos de uso, cada uno con su archivo service.go y su suite de pruebas.
- internal/http/ — adaptador HTTP: handlers (admin, audit, auth, course, docs, health, structure) y middlewares (auth, authorize, csrf, idempotency, logger, ratelimit, recoverer, request_id, tracing).
- internal/postgres/ — adaptador de persistencia, con un repositorio y su prueba por cada agregado.
- internal/cache/ — adaptador Redis (idempotency_store, rate_limiter, session_cache).
- internal/worker/ — motor de background processing (backoff, client, dlq, idempotency, metrics) y internal/worker/task, internal/worker/handler.
- internal/observability/ — proveedor de trazas OTel y métricas.
- internal/mailer/ — adaptador SMTP.

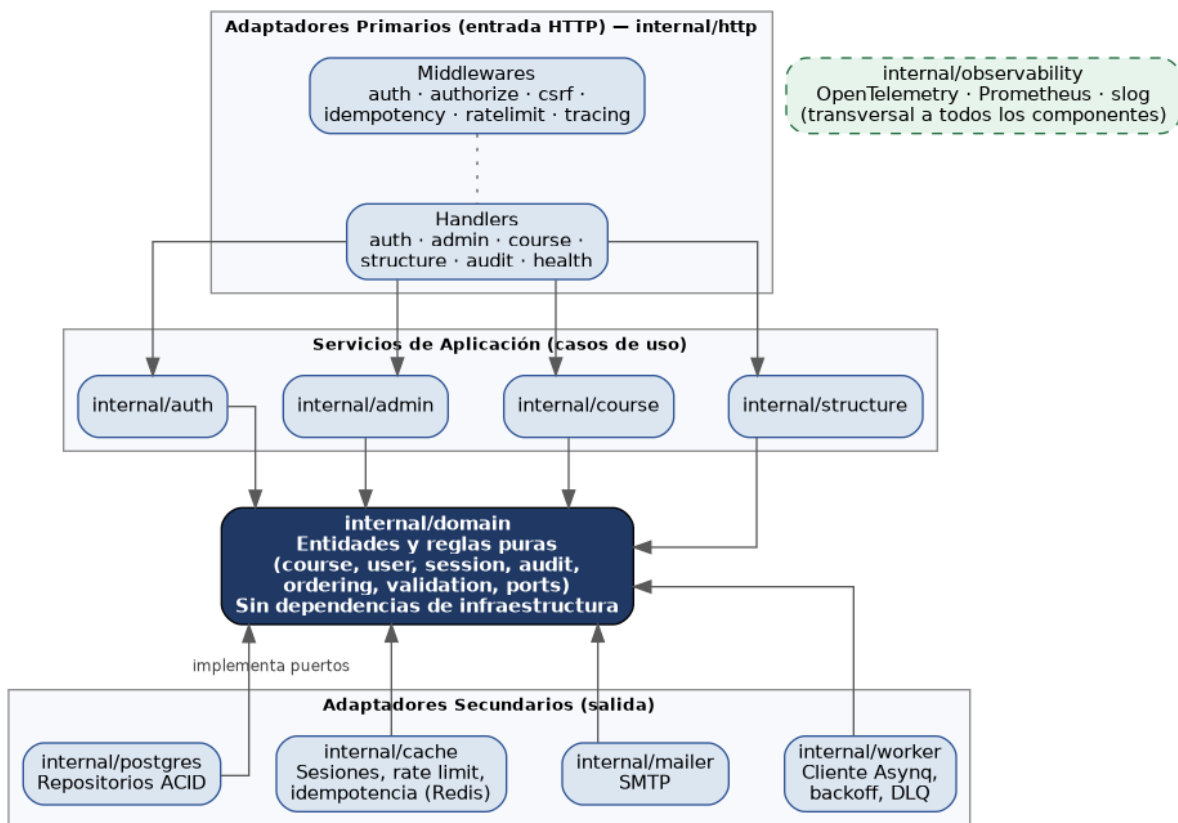


Figura 3 — Vista de Componentes del Backend, Arquitectura Hexagonal (C4 Nivel 3).

5.4 Vista de Información y Datos (Modelo Relacional e Invariantes)

El modelo de datos se gestiona mediante cinco migraciones SQL reversibles, todas verificadas en el repositorio (migrations/migrations_test.go valida el ciclo DOWN → UP → DOWN → UP):

Migración	Contenido
000001_init_schema	Tablas fundamentales, restricciones de integridad, índices B-Tree, llaves foráneas en cascada.
000002_auth_tokens	Tokens temporales de verificación de correo y restablecimiento de contraseña (hash SHA-256).
000003_academic_ordering	Restricciones de posición a UNIQUE(...) DEFERRABLE INITIALLY DEFERRED en modules, units y resources.
000004_audit_immutability	Trigger PL/pgSQL que convierte audit_logs en tabla append-only.
000005_resource_downloadable	Columna allow_download (booleana) para recursos multimedia y PDF.

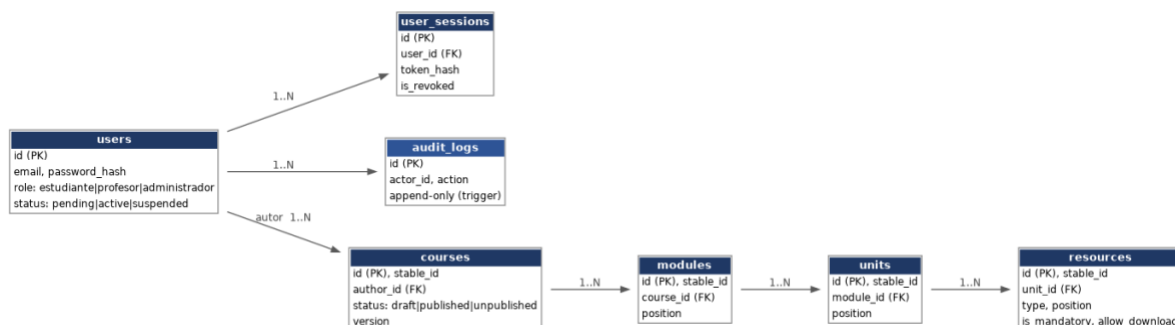


Figura 4 — Modelo Entidad-Relación simplificado (entidades centrales de la Entrega 1).

5.5 Vista de Despliegue e Infraestructura

El despliegue local se orquesta íntegramente mediante Docker Compose v2. Las imágenes Dockerfile.api y Dockerfile.worker implementan el patrón multi-stage (builder en golang:1.24-alpine, runtime en alpine:3.20 con usuario sin privilegios), logrando imágenes de menos de 30 MB.

5.6 Vista Dinámica: Flujos Críticos de Ejecución

Se documentan los tres flujos de secuencia verificados en la Entrega 1.

Flujo A — Registro, Verificación y Sesión

Registro público con verificación por correo, activación de cuenta, inicio de sesión y revocación. Verificado mediante 110 aserciones automáticas sobre la colección Postman de Identidad.

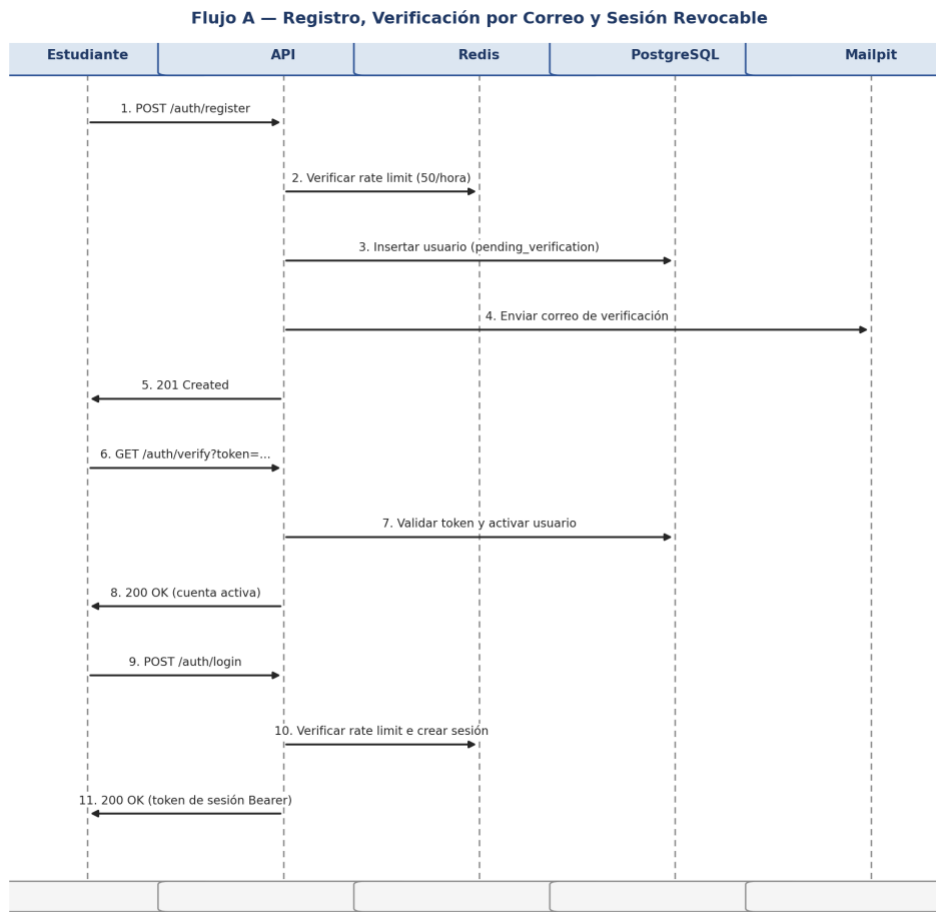


Figura 5 — Flujo A: Registro, verificación por correo y sesión revocable.

Flujo B — Autoría y Publicación de Curso

Creación de un curso en borrador, construcción de la jerarquía, validación estructural acumulativa, publicación de una versión inmutable y rechazo de ediciones posteriores. Verificado mediante 60 aserciones sobre la colección Postman de Autoría.

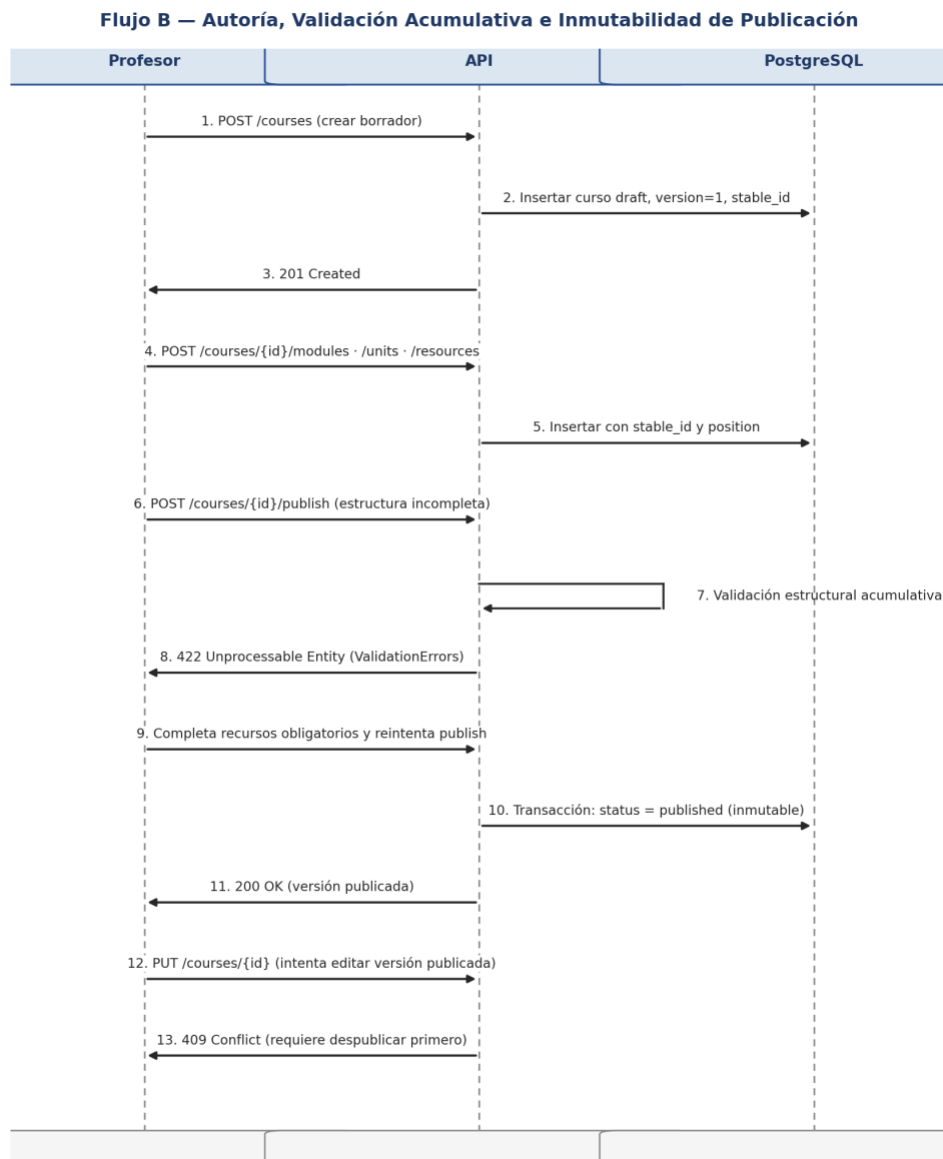


Figura 6 — Flujo B: Autoría, validación acumulativa e inmutabilidad de publicación.

Flujo C — Procesamiento Asíncrono, Reintentos y DLQ

Encolado de una tarea, fallos transitorios con reintentos de backoff exponencial, derivación a Dead-Letter Queue tras el tercer fallo, y manejo idempotente de una entrega duplicada. Verificado mediante el script `make demo-segment4` y las pruebas de `internal/worker`.

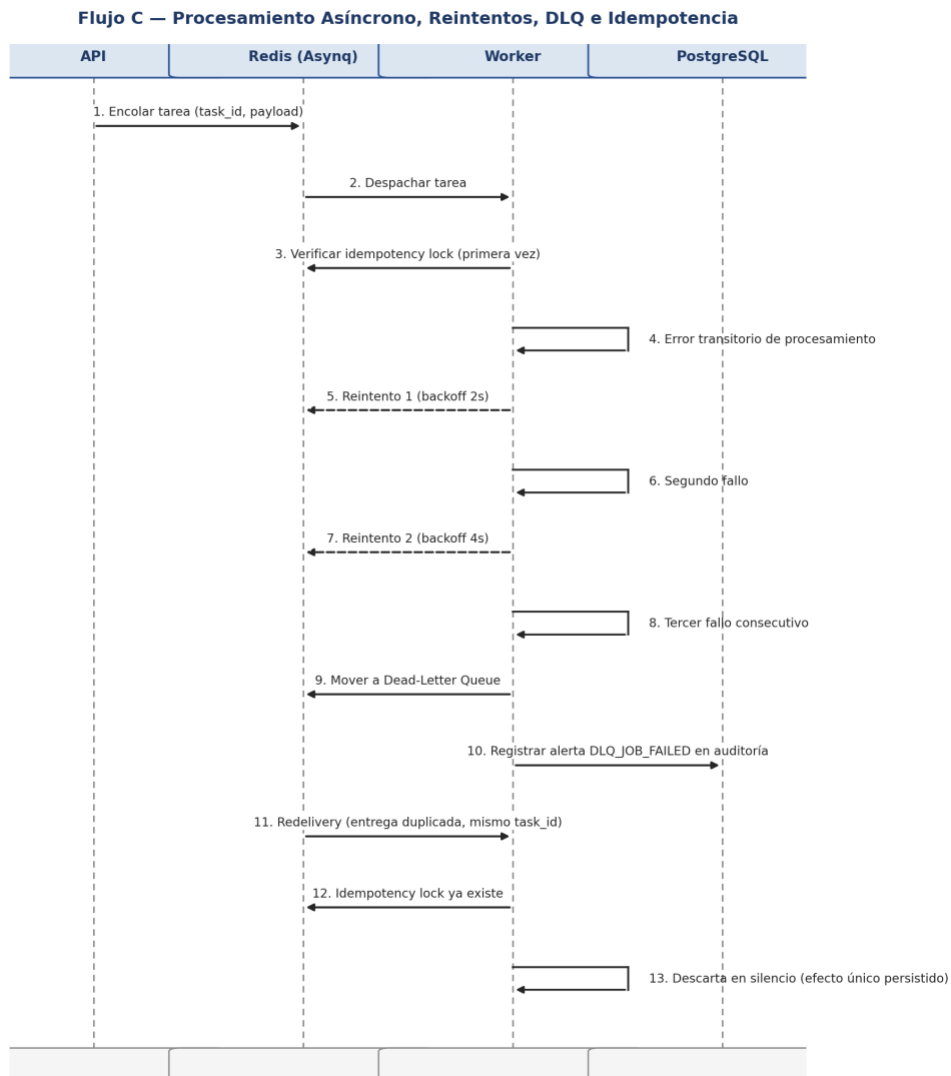


Figura 7 — Flujo C: Procesamiento asíncrono, reintentos, DLQ e idempotencia.

6. Estrategia de Seguridad, Gobierno y Cumplimiento

La plataforma aplica los principios de Seguridad por Diseño y Menor Privilegio, en un modelo de cinco capas: hardening de red y cabeceras, control de tasa, autenticación y sesiones, control de acceso (RBAC) y pista de auditoría inmutable.

- **Matriz de control de acceso:** verificación granular en `internal/http/middleware/authorize.go`; un profesor solo puede editar sus propios cursos en borrador; endpoints `/api/v1/admin/*` y `/api/v1/audit/*` restringidos al rol administrador.
- **Protección contra amenazas web comunes:** verificación de Origin/Referer en métodos mutativos (Sección 3.3.1); consultas parametrizadas en `database/sql`; filtrado de texto libre contra XSS almacenado.
- **Protección de auditoría:** cualquier intento de alteración o borrado de `audit_logs` dispara una excepción a nivel de base de datos.

7. Estrategia de Resiliencia, Concurrencia y Recuperación ante Desastres (DRP)

- **Concurrencia en API REST:** Go maneja cada conexión HTTP en una goroutine independiente (~2KB de stack inicial).
- **Pool de conexiones a base de datos:** 25 abiertas, 25 ociosas, 5 minutos de vida máxima.
- **Tolerancia a particiones:** los middlewares y workers reintentan la conexión a PostgreSQL/Redis antes de declarar fallos irre recuperables.
- **Recuperación ante desastres:** la infraestructura se reconstruye con docker compose up -d y make seed; los esquemas se autogestionan con migraciones reversibles verificadas en CI (DOWN → UP → DOWN → UP).

8. Estrategia de Observabilidad, Telemetría y Monitoreo

El sistema implementa OpenTelemetry para correlacionar trazas, métricas y registros. Cada log de log/slog sigue formato JSON estructurado con request_id y trace_id.

8.1 Métricas Prometheus Expuestas

- API REST (/api/v1/metrics, puerto 8080): http_requests_total, http_request_duration_seconds, http_requests_in_flight.
- Worker Engine (/metrics, puerto 9090): worker_jobs_processed_total, worker_jobs_failed_total, worker_job_execution_duration_seconds.

8.2 Integración Continua

El pipeline de CI (.github/workflows/ci.yml) ejecuta, en cada push y pull request: golangci-lint y Spectral (lint de OpenAPI), build de cmd/api y cmd/worker, migraciones sobre una base de datos limpia, y la suite de pruebas unitarias (go test -v ./...).

9. Estado de Implementación — Entrega 1

El milestone "Entrega 1 - Backend y Workers" agrupa 30 issues, todos completados. La siguiente matriz resume el estado de cada frente funcional y su evidencia de verificación.

Frente funcional	Issues	Evidencia de verificación
Infraestructura y CI	#1–#4, #7	Pipeline de CI (lint, build, migraciones, pruebas); Docker Compose; esqueleto OpenAPI 3.1.
Workers asíncronos	#5, #6, #8	internal/worker (backoff, DLQ, idempotencia) — pruebas unitarias y script make demo-segment4.
Identidad y sesiones	#9–#11	110 aserciones — colección Postman de Identidad (100% éxito).
Administración y seguridad transversal	#12–#14	46 aserciones — colección Postman de Administración (100% éxito).
Dominio académico y publicación	#15–#20	60 aserciones — colección Postman de Autoría (100% éxito).
Observabilidad	#21	internal/observability — trazas OTel, métricas Prometheus, logs JSON correlacionados.
QA y datos sintéticos	#22–#28	Plan de pruebas, datos sintéticos deterministas, 3 colecciones Postman, reporte E2E consolidado.
Documentación	#29, #30	README, guía de despliegue, evidencia y guion de demo.

9.1 Métricas de Verificación Automatizada

Desglose por colección Postman/Newman, ejecutada contra el sistema desplegado en Docker Compose (cero mocks):

Colección	Peticiones	Aserciones	Éxito
Identidad (#24)	59	110	100%
Administración (#25)	23	46	100%
Autoría de Cursos (#26)	30	60	100%
Total consolidado	112	216	100%

Adicionalmente, el repositorio contiene 37 archivos de prueba unitaria e integración (internal/auth, internal/course, internal/structure, internal/domain, internal/http, internal/postgres, internal/worker, internal/observability), ejecutados automáticamente en CI mediante go test -v ./..., y las 5 migraciones del esquema fueron verificadas de forma reversible (DOWN → UP → DOWN → UP) en migrations/migrations_test.go.

10. Plan de Evolución Arquitectónica y Roadmap Técnico

El proyecto se organiza en 4 a 5 entregas a lo largo de 13 semanas. La Entrega 1 (documentada en las secciones anteriores) está completa; las siguientes tres fases distribuyen el trabajo restante según sus dependencias funcionales: la inscripción de estudiantes es prerequisite del progreso, y el progreso a su vez es prerequisite de la emisión de insignias.

Plan de Entregas del Proyecto (13 semanas)

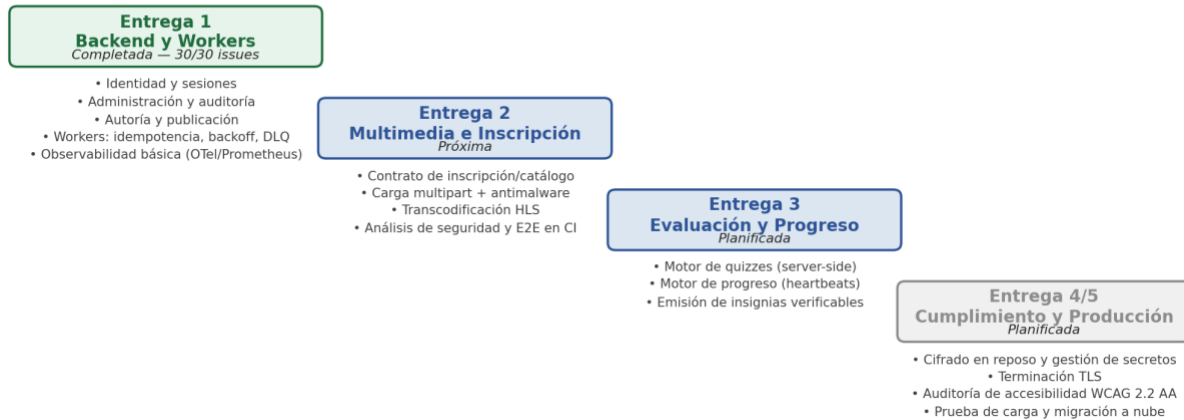


Figura 8 — Plan de entregas del proyecto.

10.1 Entrega 2 — Multimedia e Inscripción

Construye sobre la infraestructura de workers y el dominio de cursos ya implementados en la Entrega 1.

- Contrato y modelo de inscripción/catálogo: inscripción, retiro y reinscripción conservando progreso y resultados.
- Carga multipart directa a MinIO con URLs prefirmadas, verificación de checksum SHA-256 y MIME real.
- Escaneo antimalware de cargas (ClamAV).
- Transcodificación asíncrona a HLS mediante ffmpeg, integrada al worker existente.
- Incorporación de un paso de análisis de seguridad estático (SAST) y de la suite E2E al pipeline de CI.
- Prueba de carga inicial contra los flujos ya contruidos en la Entrega 1 (no depende de trabajo nuevo).

10.2 Entrega 3 — Evaluación Académica y Progreso

- Motor de evaluación de quizzes en servidor (Quiz Key Secrecy): guardado parcial, calificación server-side, intentos idempotentes.

- Motor de progreso mediante heartbeats y permanencia mínima, con rechazo y auditoría de porcentajes enviados por el cliente — depende de la inscripción de la Entrega 2.
- Emisión de insignias verificables con URL pública, sin exponer el correo del estudiante — depende del motor de progreso.

10.3 Entrega 4/5 — Cumplimiento, Seguridad y Producción

- Cifrado en reposo para PostgreSQL y MinIO, y gestión externa de secretos.
- Terminación TLS 1.2+ mediante un proxy inverso en la topología de despliegue.
- Auditoría automatizada de accesibilidad (WCAG 2.2 AA), una vez construido el frontend de consumo.
- Prueba de carga completa (2.000 usuarios concurrentes) y ejercicio de recuperación ante desastres cronometrado.
- Migración de Docker Compose a infraestructura como código (Terraform) sobre AWS/GCP: balanceador de carga, cómputo elástico, base de datos Multi-AZ y CDN.

11. Glosario de Términos

- **Asynq**: librería de procesamiento asíncrono y colas de tareas distribuidas en Go respaldada por Redis.
- **Dead-Letter Queue (DLQ)**: cola donde se almacenan las tareas que fallaron repetidamente tras agotar la cuota de reintentos.
- **ETag**: cabecera HTTP que representa la huella digital de un recurso para validación de caché condicional.
- **HLS (HTTP Live Streaming)**: protocolo de streaming de video adaptativo basado en segmentos HTTP y listas m3u8.
- **Idempotencia**: propiedad por la cual una operación produce el mismo resultado sin importar cuántas veces se ejecute con los mismos parámetros.
- **Monolito Modular**: patrón donde el sistema se compila y ejecuta como una unidad, pero su diseño interno está desacoplado en módulos con límites claros.
- **OpenTelemetry (OTel)**: estándar de observabilidad independiente de proveedores para trazas, métricas y logs.
- **RBAC**: control de acceso basado en la asignación de permisos según el rol organizacional del usuario.
- **RPO / RTO**: máxima pérdida de datos tolerable y duración máxima tolerable para restaurar el servicio tras un incidente.
- **SAST**: Static Application Security Testing — análisis estático automatizado del código fuente en busca de vulnerabilidades.
- **Stable ID**: identificador inmutable de una entidad de aprendizaje que permanece igual a lo largo de todas las versiones publicadas del curso.
- **WCAG 2.2 AA**: Web Content Accessibility Guidelines, nivel AA — criterios de accesibilidad web exigidos por el enunciado del proyecto.

12. Referencias

Documentos y artefactos consultados para elaborar este informe:

- Pliego de Especificaciones del proyecto — docs/2026-20 proyecto-plataforma-mooc (2).pdf
- docs/PROJECT_KEY_ASPECTS.md — guardrails y requisitos declarados por el equipo.
- docs/PLAN_DE_PRUEBAS_ETAPA.md — plan de pruebas de la Entrega 1.
- docs/e2e/REPORTE_E2E_IDENTIDAD_Y_AUTORIA.md — detalle petición por petición de las 216 aserciones automáticas.
- docs/e2e/REPORTE_BUGS_Y_CALIDAD_ETAPA.md — certificación de calidad y taxonomía de severidad.
- docs/DATABASE_DESIGN.md y docs/DOMINIO_ACADEMICO_IMPLEMENTACION.md — diseño de datos e implementación del dominio académico.
- docs/GUIA_DE_DESPLIEGUE.md — procedimiento de despliegue y carga de datos sintéticos.
- .github/workflows/ci.yml — definición del pipeline de integración continua.
- Milestone "Entrega 1 - Backend y Workers" e Issues #1–#30 del repositorio.
- Repositorio: github.com/ISIS4426-2026/Plataforma-MOOC, rama main, commit f8450cc1a35975ad2801ee180de7f1e8cb4f21df.